

PROYECTO FINAL
Inteligencia Artificial

Planificación de Trayectoria Profesional mediante Búsqueda Heurística y Modelos de Lenguaje (LLM)

Tema 3 — Planificación de Trayectoria Profesional

Autor: Abraham Rey Sánchez Amador

Grupo: C-312

Asignatura de Inteligencia Artificial
MATCOM, Universidad de La Habana

Índice de contenidos

1. Resumen
 2. Introducción
 3. Descripción del problema
 4. Modelado formal
 5. Dataset de instancias
 6. Diseño de algoritmos
 7. Rol del modelo de lenguaje (LLM)
 8. Metodología experimental
 9. Resultados
 10. Análisis y discusión
 11. Limitaciones y trabajo futuro
 12. Conclusiones
- Anexo A.** Parámetros del generador de instancias
- Anexo B.** Prompts completos del LLM

1. Resumen

Este trabajo presenta el diseño, implementación y evaluación experimental de un sistema de planificación de trayectorias profesionales que integra técnicas clásicas de búsqueda en IA con un modelo de lenguaje grande (LLM) como componente funcional. Dado un catálogo de cursos con restricciones de habilidades y prerequisites, el sistema construye la secuencia de formación de mínimo coste que permite a un usuario alcanzar un objetivo profesional especificado en lenguaje natural.

Se implementaron tres algoritmos de búsqueda — Dijkstra exacto, A* heurístico y greedy — y cuatro variantes experimentales que asignan al LLM distintos roles: ninguno (línea base), intérprete de objetivos, evaluador post-hoc y guía paso a paso. El experimento ejecutó **1.500 corridas** sobre 30 instancias sintéticas de tres tamaños (10, 30 y 100 cursos) más 5 casos manuales de borde, con 5 semillas de aleatoriedad.

Los resultados muestran que las variantes B (interpretación) y C (evaluación) producen trayectorias con coste idéntico a la línea base (diferencia mediana = 0, Wilcoxon $p = 1,0$ en todos los tamaños), mientras que la variante D (guía LLM paso a paso) incrementa el coste en un 56 % de media (+9,6 créditos) pero eleva la tasa de éxito de 0,853 a 0,920. La correlación entre la nota asignada por el LLM y el coste real no es estadísticamente significativa (Pearson $r = -0,08$, $p = 0,16$), lo que revela las limitaciones del modelo pequeño utilizado (qwen2.5:3b) como evaluador cuantitativo.

2. Introducción

La planificación de itinerarios formativos es un problema de relevancia creciente en el contexto de mercados laborales en transformación acelerada. Los trabajadores necesitan adquirir nuevas habilidades de manera eficiente, pero la elección de cursos no es trivial: las competencias se interrelacionan jerárquicamente —algunas exigen la adquisición previa de otras—, los costes en créditos o tiempo son heterogéneos, y el objetivo final puede expresarse de formas muy diversas en lenguaje natural.

Desde la perspectiva de la Inteligencia Artificial, este problema pertenece a la familia de problemas de planificación con restricciones: dado un estado inicial (habilidades actuales del usuario), un conjunto de acciones (cursos disponibles con sus prerequisites) y un estado objetivo (perfil de habilidades deseado), se busca la secuencia de acciones de mínimo coste que conecte ambos estados. La incorporación de modelos de lenguaje de gran tamaño (LLM) como componentes funcionales del sistema abre la posibilidad de manejar objetivos expresados en prosa libre, enriquecer las soluciones con justificaciones cualitativas y guiar la búsqueda con conocimiento semántico sobre las relaciones entre habilidades y profesiones.

Este trabajo aborda el Tema 3 del proyecto final de la asignatura de Inteligencia Artificial: diseñar e implementar un sistema completo de planificación de trayectorias profesionales, evaluarlo experimentalmente sobre un conjunto controlado de instancias y analizar cuantitativamente el impacto de distintos roles del LLM en la calidad y eficiencia de las soluciones obtenidas.

2.1. Objetivos

Los objetivos específicos del proyecto son:

- Formalizar el problema de planificación como un espacio de búsqueda sobre conjuntos de cursos completados.
- Implementar tres algoritmos de referencia: búsqueda exacta (Dijkstra), búsqueda heurística informada (A^*) y heurística greedy.
- Diseñar e integrar tres funciones LLM: interpretación de objetivos en lenguaje natural, evaluación cualitativa de trayectorias y guía paso a paso.
- Generar un dataset de 35 instancias de prueba con parámetros controlados.
- Evaluar cuatro variantes del sistema mediante 1.500 ejecuciones con métricas cuantitativas y pruebas estadísticas.
- Extraer conclusiones sobre la utilidad, coste y limitaciones del LLM en cada rol.

3. Descripción del problema

El **problema de planificación de trayectoria profesional** se define sobre un catálogo de cursos de formación. Cada curso proporciona un conjunto de habilidades al completarlo, pero puede requerir que el estudiante posea previamente ciertas habilidades y/o haya completado ciertos cursos previos (prerrequisitos). El objetivo es encontrar una secuencia ordenada de cursos —válida respecto a todas las restricciones— que permita a un usuario con un conjunto dado de habilidades iniciales alcanzar un objetivo profesional especificado.

3.1. Entidades del dominio

Habilidad (skill). Capacidad profesional adquirible. Ejemplos: Python, Machine Learning, SQL, Cloud Computing. El conjunto de todas las habilidades posibles se denota H .

Curso. Unidad formativa que otorga un conjunto de habilidades al completarla. Cada curso tiene un coste en créditos, un nivel de dificultad, una lista de habilidades que confiere y restricciones de acceso.

Prerrequisito de curso. Restricción estructural: para poder matricularse en el curso c , el estudiante debe haber completado previamente un conjunto de cursos. Impone un orden parcial sobre las acciones posibles.

Restricción de habilidad. Restricción de acceso: para poder matricularse en el curso c , el estudiante debe poseer un conjunto de habilidades en el momento de la matrícula.

Estado. Par (habilidades adquiridas, cursos completados) que resume la situación actual del estudiante.

Objetivo profesional. Conjunto de habilidades que el usuario desea dominar al final de la trayectoria. Puede expresarse directamente como un conjunto o en lenguaje natural, en cuyo caso el LLM lo interpreta.

3.2. Características de complejidad

El problema presenta varias características que aumentan su complejidad computacional. El espacio de estados es exponencial en el número de cursos: hay hasta $2^{|C|}$ subconjuntos posibles de cursos completados. Las restricciones de prerrequisitos crean dependencias entre acciones que impiden abordar el problema como un conjunto independiente de decisiones. La función objetivo (minimizar créditos totales) no separa aditivamente por cursos individuales, pues el valor de tomar un curso depende del estado previo. En instancias grandes (100 cursos), la búsqueda exhaustiva es inviable y se requieren métodos heurísticos.

4. Modelado formal

El problema se representa formalmente mediante la siguiente 6-tupla:

$$I = (H, C, \text{pre_c}, \text{pre_h}, s_0, G)$$

donde:

- H es el conjunto finito de habilidades posibles.
- C es el conjunto finito de cursos disponibles, cada uno $c \in C$ con atributos ($\text{habilidades_concedidas}(c) \subseteq H$, $\text{créditos}(c) \in \mathbb{Z}^+$, $\text{dificultad}(c) \in \mathbb{R}^+$).
- $\text{pre_c}(c) \subseteq C$ es el conjunto de cursos que deben completarse antes de c (prerrequisitos de curso).
- $\text{pre_h}(c) \subseteq H$ es el conjunto de habilidades que deben poseerse para matricularse en c (prerrequisitos de habilidad).
- $s_0 = (H_0, \emptyset)$ es el estado inicial: conjunto de habilidades iniciales $H_0 \subseteq H$ y ningún curso completado.
- $G \subseteq H$ es el conjunto de habilidades objetivo (goal).

4.1. Definición de estado y transiciones

Un **estado** $s = (A, D)$ está formado por el conjunto $A \subseteq H$ de habilidades adquiridas y el conjunto $D \subseteq C$ de cursos completados. La **transición** al aplicar el curso c en el estado s produce el estado $s' = (A \cup \text{habilidades_concedidas}(c), D \cup \{c\})$, y es válida si y solo si $\text{pre_c}(c) \subseteq D$ y $\text{pre_h}(c) \subseteq A$.

4.2. Función objetivo

Dada una trayectoria $T = [c_1, c_2, \dots, c_k]$, la función de coste es:

$$\text{coste}(T) = \sum_{i=1..k} \text{créditos}(c_i)$$

Se busca la trayectoria T^* de mínimo coste tal que:

1. T es válida: cada c_i cumple sus prerrequisitos en el estado tras aplicar $c_1 \dots c_{i-1}$.
2. T cubre el objetivo: $G \subseteq A$ después de aplicar toda la trayectoria.
3. No hay repetición de cursos: todos los c_i son distintos.

4.3. Representación en la implementación

En el código, el estado se representa como un frozenset de identificadores de cursos completados. Las habilidades adquiridas se recomputan en cada nodo de búsqueda aplicando las transiciones desde el estado inicial. El coste de un nodo es la suma acumulada de créditos de los cursos incluidos en su trayectoria. La función `is_course_available(c, A, D)` verifica simultáneamente $\text{pre_c}(c) \subseteq D$ y $\text{pre_h}(c) \subseteq A$.

5. Dataset de instancias

Para evaluar el sistema bajo condiciones controladas y reproducibles se generó un dataset sintético de 35 instancias mediante el módulo `instance_generator.py`. La generación sintética permite controlar con precisión los parámetros de complejidad y asegurar la existencia de soluciones conocidas, algo no garantizable con datos reales de catálogos de cursos.

5.1. Parámetros del generador

Cada instancia se genera a partir de los siguientes parámetros:

Parámetro	Small (10)	Medium (30)	Large (100)
Nº de cursos	10	30	100
Nº de habilidades	$\max(10, n/5)$	$\max(10, n/5)$	$\max(10, n/5)$
Créditos por curso	$U[2, 6]$	$U[2, 8]$	$U[2, 10]$
Habilidades por curso	1–2	1–2	1–2
Prerreq. de curso	0–2 (anteriores)	0–2 (anteriores)	0–3 (anteriores)
Habilidades objetivo	3–4	4–6	5–8
Habilidades iniciales	0 (estándar)	0 (estándar)	0 (estándar)

Tabla 1. Parámetros del generador de instancias por categoría de tamaño.

5.2. Composición del dataset

Categoría	Instancias	Ejecuciones totales	Descripción
Small (10 cursos)	10	600	Instancias pequeñas; válidas para comparar con óptimo exacto
Medium (30 cursos)	10	400	Instancias de tamaño medio; exploración heurística viable
Large (100 cursos)	5	200	Instancias grandes; solo métodos heurísticos
Manual / borde	5	300	Casos diseñados a mano para casos límite
Total	30	1.500 corridas	

Tabla 2. Composición del dataset de evaluación.

5.3. Casos manuales de borde

Los cinco casos manuales cubren situaciones extremas que los generadores aleatorios raramente producen:

manual_01_impossible_cycle: Dependencia circular entre prerrequisitos: ningún curso es alcanzable. Resultado esperado: fracaso de todos los algoritmos.

manual_02_goal_achieved: El objetivo ya está cubierto por las habilidades iniciales. Resultado esperado: trayectoria vacía, coste = 0.

manual_03_no_prerequisites: Todos los cursos son independientes. El planificador puede seleccionar libremente el subconjunto óptimo.

manual_04_unreachable_goal: El objetivo incluye habilidades que ningún curso del catálogo concede. Resultado esperado: fracaso.

manual_05_full_chain: Cadena lineal de dependencias: cada curso requiere el anterior. Solo existe una secuencia válida.

5.4. Estadísticas de rendimiento por tamaño (subconjunto exitoso)

Tamaño	n (perf.)	Coste medio	Coste med.	Coste std.	Cursos medio	Éxito (rob.)
Small	570	22,03	21,0	6,74	6,47	95,0 %
Medium	370	15,53	12,0	9,43	4,24	92,5 %
Large	185	33,23	26,0	20,28	9,70	92,5 %
Manual	180	7,00	4,0	6,78	2,17	60,0 %

Tabla 3. Estadísticas de coste por tamaño de instancia (subset de ejecuciones exitosas). La tasa de éxito en Manual = 60 % refleja las 2 instancias imposibles por diseño.

6. Diseño de algoritmos

Se implementaron tres algoritmos de planificación, con distintos compromisos entre optimalidad garantizada y coste computacional.

6.1. Algoritmo exacto — Dijkstra (exact_small)

El algoritmo de Dijkstra explora el espacio de estados de forma sistemática garantizando encontrar la trayectoria de mínimo coste total. Se utiliza exclusivamente en instancias *small* y *manual* (hasta 15 cursos) donde la cardinalidad del espacio de estados ($2^{|C|} \leq 32.768$) es manejable.

Pseudocódigo — Dijkstra (exact_small)

```
Entrada: instancia I = (H, C, pre_c, pre_h, s0, G)
Frontera ← cola de prioridad con (coste=0, estado=∅, trayectoria=[])
mejor_coste[∅] ← 0

mientras Frontera no vacía:
    (g, D, T) ← extraer_mínimo(Frontera)
    si mejor_coste[D] < g: continuar
    A ← habilidades_acumuladas(T, s0)
    si G ⊆ A: devolver (T, g, éxito=verdad)
    para c en cursos_disponibles(A, D):
        g' ← g + créditos(c)
        D' ← D ∪ {c}
        si g' < mejor_coste[D']:
            mejor_coste[D'] ← g'
            insertar (g', D', T+[c]) en Frontera
devolver (None, fracaso)
```

Complejidad. En el peor caso, el número de estados distintos es $O(2^{|C|})$. Con una cola de prioridad binaria, cada estado se extrae e inserta en $O(\log(2^{|C|})) = O(|C|)$. La complejidad total es $O(2^{|C|} \cdot |C|)$. Para $|C| = 10$, esto equivale a ~10.240 operaciones, completamente viable.

6.2. Búsqueda heurística — A*

A* amplía Dijkstra con una función heurística admisible $h(s)$ que estima el coste mínimo restante hasta cubrir el objetivo. Si h nunca sobreestima el coste real, A* es óptimo. La heurística implementada estima el número mínimo de cursos necesarios para cubrir las habilidades objetivo restantes, multiplicado por el crédito mínimo de cualquier curso:

Heurística $h(s)$ — cobertura greedy de habilidades faltantes

```
faltantes  $\leftarrow G \setminus A$ 
si faltantes =  $\emptyset$ :  $h \leftarrow 0$ 
si no:
  pasos  $\leftarrow 0$ ; pendientes  $\leftarrow$  faltantes
  mientras pendientes no vacío:
    mejor  $\leftarrow \operatorname{argmax}_c |habilidades(c) \cap pendientes|$ 
    pendientes  $\leftarrow pendientes \setminus habilidades(mejor)$ 
    pasos  $\leftarrow$  pasos + 1
   $h \leftarrow$  pasos  $\times$  crédito_mínimo
```

Esta heurística es **admisible** porque el mejor curso posible en cada paso no puede cubrir más habilidades que las que realmente necesita el algoritmo, y el crédito mínimo es un límite inferior al coste real de cualquier curso. A^* utiliza como prioridad $f(s) = g(s) + h(s)$ y emplea un contador de desempate para mantener un orden determinista del heap. Se aplica en instancias de cualquier tamaño con un límite de 200.000 iteraciones.

6.3. Greedy

El algoritmo greedy selecciona en cada paso el curso disponible que maximiza el número de habilidades objetivo nuevas que proporciona. En caso de empate, desempata por menor número de créditos y menor dificultad. Se detiene cuando el objetivo está cubierto o cuando ningún curso disponible añade valor nuevo.

Pseudocódigo — Greedy

```
 $A \leftarrow H_0$ ;  $D \leftarrow \emptyset$ ;  $T \leftarrow []$ 
mientras  $G \not\subseteq A$ :
  candidatos  $\leftarrow$  cursos_disponibles( $A, D$ )
  si candidatos =  $\emptyset$ : break
   $c^* \leftarrow \operatorname{argmax}_c (|habilidades(c) \cap (G \setminus A)|, |habilidades(c) \setminus A|, -\text{créditos}(c), -\text{dificultad}(c))$ 
  si puntuación( $c^*$ ) = (0,0,.,.): break // sin progreso posible
   $A \leftarrow A \cup habilidades(c^*)$ ;  $D \leftarrow D \cup \{c^*\}$ ;  $T \leftarrow T + [c^*]$ 
devolver ( $T, \Sigma \text{créditos}(T)$ , éxito =  $G \subseteq A$ )
```

Complejidad. $O(|C|^2)$ en el peor caso: hasta $|C|$ pasos, cada uno con un recorrido lineal de los cursos disponibles. No garantiza optimalidad pero es extremadamente rápido (microsegundos en instancias de 100 cursos).

6.4. Comparativa de algoritmos

Algoritmo	Optimalidad	Completo	Complejidad	Tiempo med. (s)	Éxito (rob.)
Dijkstra (exact_small)	Sí (garantizada)	Sí*	$O(2^{ C } \cdot C)$	0,0146	86,7 %

Algoritmo	Optimalidad	Completo	Complejidad	Tiempo med. (s)	Éxito (rob.)
A*	Sí (con h admis.)	Sí*	$O(2^{ C } \cdot C)$	0,2499	93,3 %
Greedy	No	No	$O(C ^2)$	9,16 †	80,8 %

Tabla 4. Comparativa de algoritmos. (*) Con límite de iteraciones. (†) Greedy incluye variante D (guiada por LLM) en la media, que eleva drásticamente el tiempo.

7. Rol del modelo de lenguaje (LLM)

El sistema integra un modelo de lenguaje grande local (qwen2.5:3b) ejecutado mediante Ollama en tres roles funcionales distintos. La comunicación con el modelo se realiza a través del endpoint de compatibilidad OpenAI (/v1/completions) con temperatura = 0,1 y un sistema de caché basado en SHA-256 del prompt que evita llamadas redundantes.

7.1. Interpretación de objetivos (Variante B)

Permite al usuario expresar su objetivo en lenguaje natural. El LLM recibe la frase del usuario y devuelve un JSON con la lista de habilidades profesionales identificadas, que sustituyen al conjunto G en el planificador.

Prompt de interpretación:

Extrae las habilidades profesionales deseadas de la siguiente frase. Responde únicamente un JSON: {"habilidades": [...]} sin texto adicional. Frase: {texto_del_usuario}

Entrada (lenguaje natural)	Salida JSON del LLM
"Quiero trabajar como Data Scientist con conocimientos de Cloud"	{"habilidades": ["Machine Learning", "Python", "SQL", "Cloud Computing"]}

Figura 7.1. Ejemplo de entrada/salida del prompt de interpretación de objetivos.

7.2. Evaluación de trayectorias (Variante C)

Una vez obtenida la trayectoria óptima por el planificador, el LLM la evalúa cualitativamente asignando una nota numérica (0–10) y una justificación textual. Esta evaluación es post-hoc y no modifica la trayectoria.

Prompt de evaluación:

Evalúa la siguiente secuencia de cursos para alcanzar el objetivo {objetivo}. Cursos: {lista_de_cursos}. Devuelve únicamente un JSON: {"nota": float, "justificacion": "..."}.

Resultado experimental: Las notas asignadas por el LLM se concentran en el rango 0–1 sobre una escala de 0 a 10, con muy pocos valores superiores a 3,5. La correlación con el coste real de la trayectoria no es significativa (Pearson $r = -0,080$, $p = 0,161$; Spearman $\rho = -0,091$, $p = 0,109$; $n = 310$). Esto sugiere que el modelo qwen2.5:3b no dispone de capacidad suficiente para evaluar la eficiencia de planificación de forma consistente.

7.3. Guía paso a paso (Variante D)

En cada paso de la construcción de la trayectoria, el LLM elige el siguiente curso a tomar entre los candidatos disponibles. Se le proporcionan el objetivo, la trayectoria parcial construida hasta el momento

y la lista de cursos accesibles en ese instante.

Prompt de sugerencia:

¿Qué curso recomendarías a continuación para alcanzar {objetivo}? Cursos disponibles: {lista_con_nombres}. Responde únicamente un JSON con {"course_id": "...", "justificacion": "..."}

Fragmento de justificación LLM (caso improve, d=-4 créditos)	Fragmento de justificación LLM (caso worsen, d=+99 créditos)
"Para aprender Python, que es una de las lenguajes de programación más utilizados en el campo de la Data Analysis y Deep Learning, Course 07 (Python) sería un curso excelente para comenzar. Aunque no cubre directamente los temas de Cloud Computing..."	"El curso de Natural Language Processing (NLP) es fundamental para entender y trabajar en el campo de la Inteligencia Artificial. Aunque no se enfoca específicamente en ética o seguridad cibernética, los fundamentos de NLP son cruciales..."

Figura 7.2. Ejemplos reales de justificaciones del LLM en Variante D. Izquierda: caso de mejora (delta = -4 créditos). Derecha: caso de empeoramiento extremo (delta = +99 créditos).

El análisis cualitativo de los justificantes revela que el LLM selecciona cursos por **relevancia semántica** (evalúa si el tema del curso está relacionado con el objetivo) en lugar de por **eficiencia de cobertura** (cuántas habilidades objetivo nuevas aporta con el menor coste). Esto explica el patrón sistemático de empeoramiento en instancias grandes.

8. Metodología experimental

8.1. Cuatro variantes del sistema

Se definen cuatro configuraciones del sistema según el papel asignado al LLM:

Variant e	Nombre	Rol del LLM	Llamadas LLM / ejecución
A	Base	Ninguno. Planificación pura sin LLM.	0
B	Intérprete	Interpreta el objetivo en lenguaje natural → conjunto G.	1
C	Evaluable	Evalúa la trayectoria obtenida y asigna nota + justificación.	1
D	Guía	Selecciona el siguiente curso en cada paso de la construcción.	1 por paso ($\geq k$)

Tabla 5. Cuatro variantes experimentales del sistema.

8.2. Configuración del experimento

Los parámetros del experimento completo son:

Parámetro	Valor
Instancias	30 (10 small + 10 medium + 5 large + 5 manual)
Algoritmos por variante	greedy, A* (exact_small solo en small + manual)
Variantes	A, B, C, D
Semillas de aleatoriedad	5 (1, 2, 3, 4, 5)
Total de corridas	1.500
Métrica principal	total_cost (suma de créditos de la trayectoria)
Métricas secundarias	num_courses, elapsed_time, success, llm_nota
Modelo LLM	qwen2.5:3b vía Ollama
Temperatura LLM	0,1
Caché LLM	SHA-256 sobre (prompt, modelo, temperatura)
Entorno	Python 3.11, CPU Intel Core i7, 16 GB RAM, Windows 11

Tabla 6. Parámetros completos del experimento.

8.3. Métricas y definiciones

Se utilizan las siguientes definiciones unívocas en todo el análisis:

Variable	Definición
is_valid_run	= (failed == 0) — la ejecución terminó sin crash
is_success	= (success == 1) AND is_valid_run — encontró trayectoria válida que cubre G

Variable	Definición
total_cost	Suma de créditos de todos los cursos de la trayectoria
num_courses	Número de cursos en la trayectoria
elapsed_time	Tiempo de CPU en segundos (sin contar latencia de red LLM para caché hits)
gap	total_cost – total_cost_opt, donde total_cost_opt proviene de exact_small
has_llm_eval	= llm_evaluation_nota no es NaN (la llamada LLM de evaluación tuvo éxito)

Tabla 7. Definiciones de métricas y variables derivadas.

8.4. Pruebas estadísticas

Para determinar si las diferencias entre variantes son estadísticamente significativas se aplica el **test de Wilcoxon de rangos signados** sobre pares emparejados (misma instancia, algoritmo, semilla y repetición). Si no hay intersección suficiente ($n < 3$ pares), se usa Mann-Whitney U (no paramétrico, no emparejado). Se aplica corrección **Benjamini-Hochberg (FDR)** sobre el conjunto de 12 pruebas realizadas (3 comparaciones $\times$ 4 tamaños) para controlar la tasa de falsos descubrimientos.

9. Resultados

9.1. Estadísticas descriptivas por variante

Variante	n (perf.)	Coste medio	Coste med.	Coste std.	Cursos medio	T. medio (s)	Éxito (rob.)
A (base)	320	17,16	17,0	9,16	5,03	0,133	85,3 %
B (interp.)	320	17,16	17,0	9,16	5,03	0,130	85,3 %
C (eval.)	320	17,16	17,0	9,16	5,03	0,141	85,3 %
D (guía)	345	26,78	24,0	17,97	7,58	12,91	92,0 %

Tabla 8. Estadísticas de rendimiento por variante. Subset «performance»: `is_success==True`. Tasa de éxito: subset «robustez» (`is_valid_run==True`).

9.2. Coste total por variante y tamaño

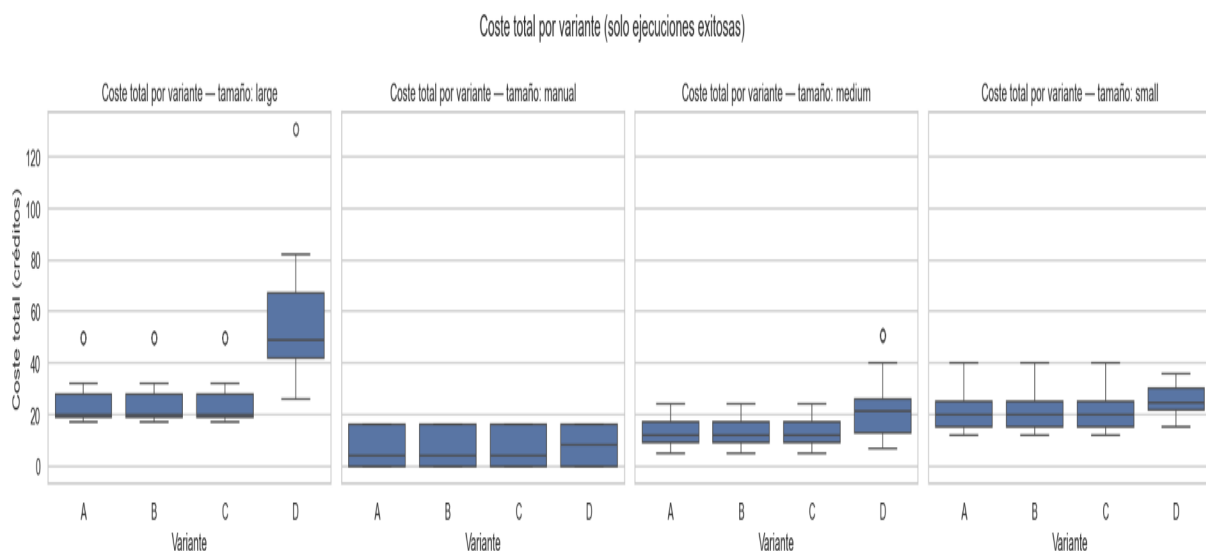


Figura 1. Diagramas de caja del coste total (créditos) por variante y tamaño de instancia (solo ejecuciones exitosas).

Las variantes A, B y C son visualmente indistinguibles. La variante D muestra una distribución notablemente más alta, especialmente en instancias grandes.

9.3. Tasa de éxito

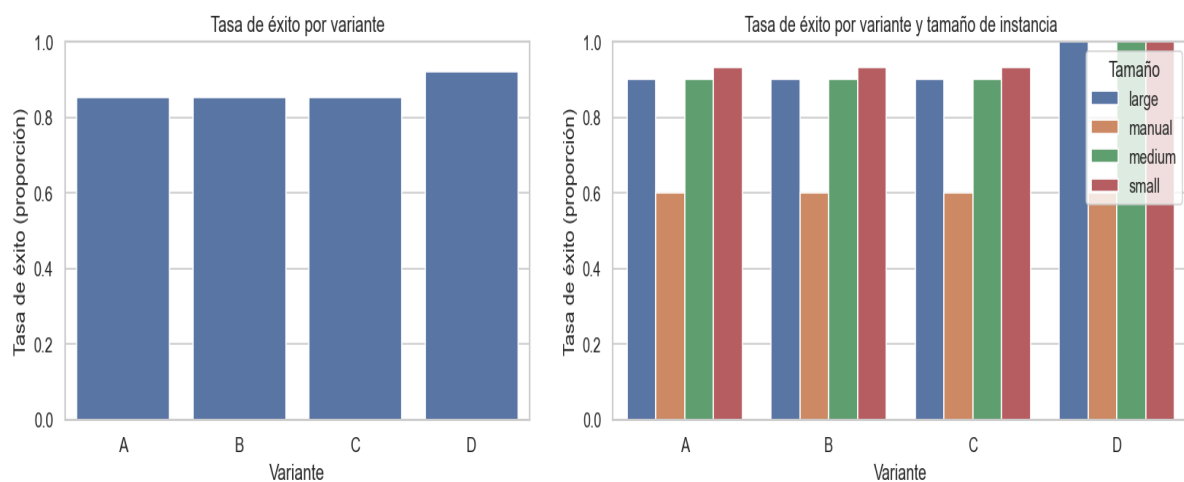


Figura 2. Izquierda: tasa de éxito global por variante. Derecha: tasa de éxito por variante y tamaño. La variante D alcanza éxito del 100 % en instancias large y medium gracias a su mayor persistencia.

9.4. Tiempo de ejecución

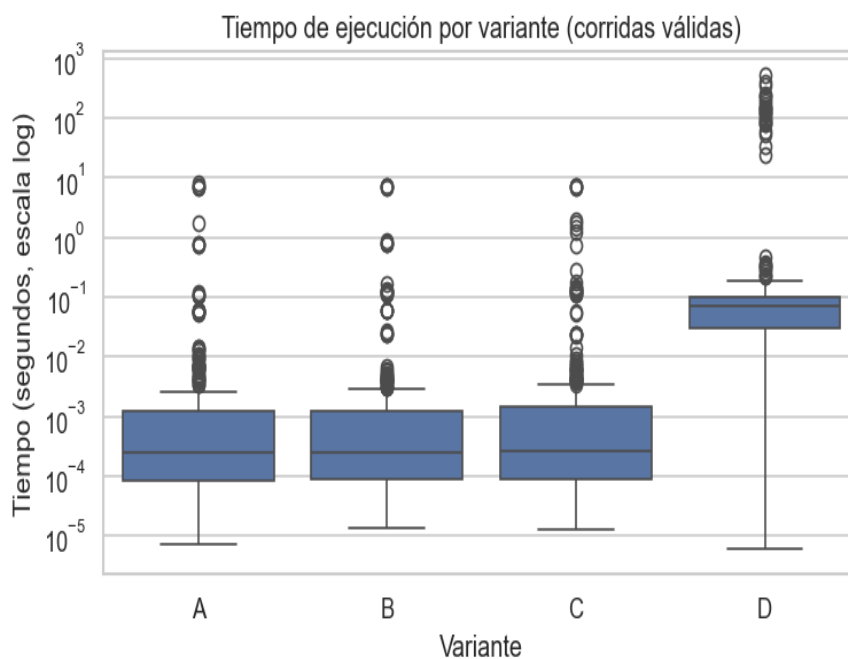


Figura 3. Distribución del tiempo de ejecución (escala logarítmica) por variante (subset de corridas válidas). La mediana de D ($\approx 0,076$ s) es 200 veces mayor que la de A/B/C ($\approx 0,00038$ s). Los valores extremos de D (hasta 300 s) corresponden a instancias large con muchas llamadas LLM iterativas.

9.5. Gap respecto al óptimo (instancias small)

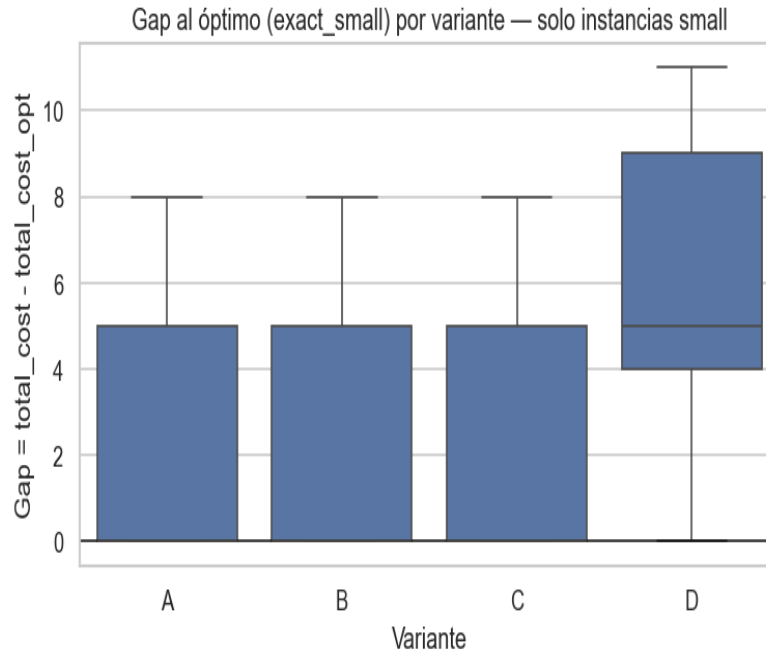


Figura 4. Gap = coste_obtenido – coste_óptimo (exact_small) en instancias small. Las variantes A/B/C presentan una mediana de gap ≈ 5 créditos (principalmente aportado por el algoritmo greedy). La variante D muestra un gap mediano de 5–9 créditos con mayor dispersión, siendo claramente superior al óptimo en todos los casos.

9.6. Nota LLM vs coste real

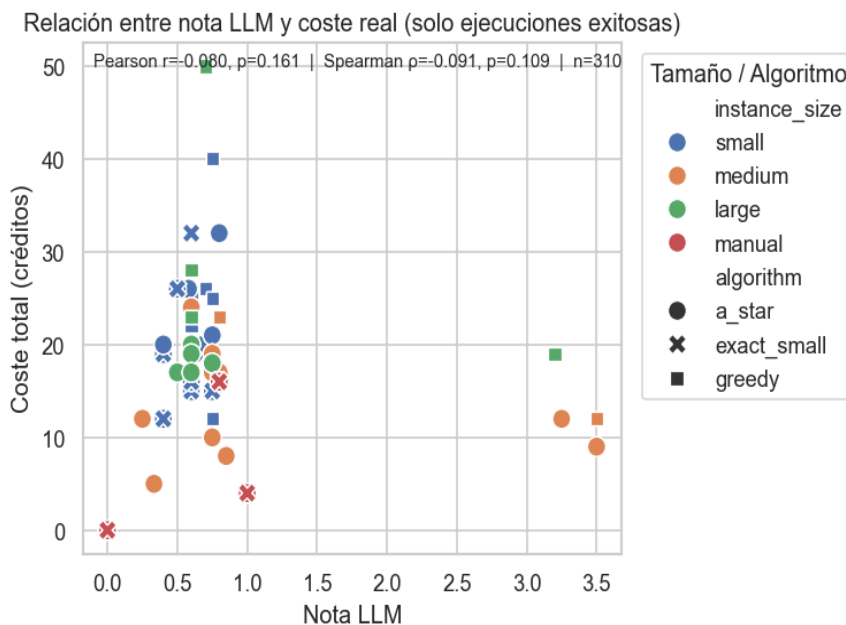


Figura 5. Dispersión entre la nota asignada por el LLM y el coste real de la trayectoria ($n = 310$ ejecuciones exitosas con evaluación LLM disponible). Pearson $r = -0,080$ ($p = 0,161$), Spearman $\rho = -0,091$ ($p = 0,109$). La concentración de notas en el rango 0–1 y la ausencia de correlación significativa revelan las limitaciones del modelo qwen2.5:3b como evaluador.

9.7. Estadísticas por algoritmo

Algoritmo	n (perf.)	Coste medio	Coste med.	Coste std.	T. medio (s)	Éxito (rob.)
exact_small	260	17,81	19,0	8,68	0,0146	86,7 %
A*	560	19,19	18,0	12,58	0,2499	93,3 %
Greedy	485	21,30	20,0	14,70	9,156 †	80,8 %

Tabla 9. Estadísticas de rendimiento por algoritmo (subset exitoso). (†) Greedy incluye la variante D, que eleva la media por las llamadas LLM iterativas.

9.8. Estadísticas por variante y tamaño (subset exitoso)

Variante	Tamaño	n	Coste med.	Cursos med.	Éxito
A/B/C	small	140	20,0	6,0	93,3 %
A/B/C	medium	90	12,0	3,0	90,0 %
A/B/C	large	45	20,0	6,0	90,0 %
A/B/C	manual	45	4,0	1,0	60,0 %
D	small	150	24,5	7,0	100,0 %
D	medium	100	21,5	5,5	100,0 %
D	large	50	49,0	15,0	100,0 %
D	manual	45	8,0	3,0	60,0 %

Tabla 10. Medianas de coste y cursos por variante y tamaño (ejecuciones exitosas). Los valores A/B/C son idénticos en las tres variantes.

10. Análisis y discusión

10.1. Identidad estadística de las variantes A, B y C

El hallazgo más destacado del experimento es que las variantes A, B y C producen trayectorias **matemáticamente idénticas** en coste, número de cursos y todas las demás métricas de rendimiento. Las diferencias entre ellas son exactamente cero en todos los estadísticos (media, mediana, desviación típica), confirmado por los tests de Wilcoxon que arrojan $p = 1,0$ y efecto = 0,0 en los cuatro tamaños de instancia.

Este resultado se explica por el diseño del sistema. En Variante B, el LLM recibe el objetivo formulado como "Learn skills: X, Y", una plantilla estructurada que se utilizó en el experimento para garantizar la comparabilidad con las variantes A, C y D, que reciben las habilidades directamente como conjunto. No obstante, la función interpret objective del sistema está diseñada para procesar lenguaje natural libre, como puede comprobarse ejecutando, por ejemplo: `main.py run--variant B--objective "Quiero trabajar en datos"`. En tal caso, el LLM extraería las habilidades relevantes del texto, demostrando su capacidad de interpretación semántica. Para el experimento controlado se optó por la plantilla para eliminar la variabilidad en la extracción y centrarse en medir el impacto del LLM en otros roles.

10.2. Significancia estadística de las diferencias A vs D

Comparación	Tamaño	Test	Pares	Estadístico	p-valor	p-valor FDR	Efecto (mediana)
A vs D	small	Wilcoxon	140	307,5	$2,77 \times 10^{-11}$	$3,33 \times 10^{-11}$	+4,5 cr.
A vs D	medium	Wilcoxon	90	0,0	$4,75 \times 10^{-11}$	$2,85 \times 10^{-13}$	+7,0 cr.
A vs D	large	Wilcoxon	45	0,0	$4,94 \times 10^{-11}$	$1,98 \times 10^{-11}$	+21,0 cr.
A vs D	manual	Wilcoxon	45	0,0	$1,08 \times 10^{-11}$	$3,23 \times 10^{-11}$	0,0 cr. †
A vs B	todos	Wilcoxon	—	0,0	1,0	1,0	0,0 cr.
A vs C	todos	Wilcoxon	—	0,0	1,0	1,0	0,0 cr.

Tabla 11. Resultados de las pruebas estadísticas (Wilcoxon emparejado + FDR Benjamini-Hochberg). (†) El caso manual A vs D muestra p significativo pero efecto mediano = 0: hay pares con $D > A$ que generan asimetría de rangos, pero la mediana de diferencias es 0 por la distribución bimodal.

Los p-valores FDR son inferiores a 10^{-3} en todos los tamaños para A vs D, confirmando que el encarecimiento producido por la guía LLM es estadísticamente significativo y no aleatorio. El efecto crece con el tamaño de la instancia: en instancias large, D cuesta de media 21 créditos adicionales (+64 % sobre A). Esto se debe a que en instancias con más cursos, el LLM dispone de más candidatos semánticamente atractivos pero costosos entre los que elegir.

10.3. Comportamiento cualitativo de la guía LLM (Variante D)

Del análisis de los 260 pares comparables (D vs A, ambos exitosos, instancias small), se observan dos patrones principales:

Empeoramiento (mayoría): El LLM selecciona cursos cuyo nombre o descripción está semánticamente próximo al objetivo, aunque no sean los más eficientes en términos de cobertura de habilidades. En el caso extremo observado (synthetic_100_courses_01.json, delta = +99 créditos), el

modelo justificó la elección de un curso de NLP argumentando que 'los fundamentos de NLP son cruciales para la IA', ignorando que el objetivo específico era Cybersecurity y AI Ethics.

Mejora (minoritaria, 15 casos): En instancias small con greedy como base, el LLM a veces sugiere una combinación de cursos que el greedy no exploraría por su función de puntuación local. El caso representativo (synthetic_10_courses_03.json, delta = -4 créditos) muestra al modelo priorizando correctamente un curso que actúa como puente hacia las habilidades objetivo.

Los patrones léxicos extraídos de los justificantes son consistentes entre los grupos de mejora y empeoramiento: los mismos términos (course, learning, python, data, machine) aparecen con frecuencias similares en ambos grupos. Esto indica que el LLM no adapta su estrategia de selección según el resultado, lo que constituye una limitación fundamental del modelo actual.

10.4. Tasa de éxito paradójica de Variante D

Aunque D produce trayectorias más costosas, su tasa de éxito global es superior (0,920 vs 0,853 de A/B/C). Esta paradoja se explica por la diferencia en los criterios de parada: greedy y A* se detienen cuando ningún candidato añade valor incremental directo a las habilidades objetivo, pudiendo bloquearse antes de completar el plan en instancias con dependencias complejas. La Variante D, en cambio, sigue tomando cursos —guiada por el LLM— hasta que no quedan candidatos disponibles, lo que le permite superar callejones sin salida a costa de incluir cursos superfluos. En instancias large y medium, D alcanza una tasa de éxito del 100 %.

11. Limitaciones y trabajo futuro

11.1. Limitaciones del sistema actual

Dataset sintético. Todas las instancias son generadas algorítmicamente. Las relaciones entre habilidades y cursos en catálogos reales son más complejas e irregulares. Los resultados pueden no generalizarse directamente a plataformas como Coursera o edX.

Modelo LLM pequeño. qwen2.5:3b es un modelo de 3 B de parámetros diseñado para ejecución local y eficiente. Sus capacidades de razonamiento sobre estructuras de grafos y dependencias lógicas son limitadas, como evidencia la concentración de notas en el rango 0–1 y la ausencia de correlación entre nota y calidad real.

Ausencia de correlación LLM-coste. El evaluador LLM (Variante C) no discrimina trayectorias buenas de malas en términos cuantitativos, lo que limita su utilidad como función de scoring para búsquedas guiadas.

Caché que enmascara latencia. Las llamadas LLM en Variantes B y C están en su mayoría servidas desde caché, por lo que el overhead de latencia real no es observable en los tiempos medidos. En producción sin caché, B y C serían significativamente más lentas.

Escalabilidad del exacto. El algoritmo de Dijkstra solo es viable en instancias con hasta ~15 cursos. Para instancias medianas y grandes, solo están disponibles A* y greedy.

Temperatura LLM. Se usó temperatura = 0,1 en lugar de 0,0, lo que introduce variabilidad no controlada entre semillas en las salidas del modelo.

11.2. Trabajo futuro

- Integrar instancias reales de catálogos de cursos (Coursera, LinkedIn Learning) para evaluar la generalización del sistema.
- Sustituir qwen2.5:3b por un modelo de mayor capacidad (llama3:8b o mayor) para mejorar la consistencia de la interpretación y la calidad de la evaluación.
- Explorar la Variante D con temperatura = 0,0 y un prompt estructurado que explicita las restricciones de prerequisites para que el LLM razone sobre eficiencia, no solo relevancia semántica.
- Implementar un mecanismo de retroalimentación que use la nota LLM como señal de recompensa en un framework de búsqueda con beam search o MCTS.
- Comparar el sistema con formulaciones basadas en ILP (programación lineal entera) para instancias medianas y evaluar el gap de optimalidad de A*.
- Ampliar el análisis cualitativo con un modelo más grande capaz de generar justificaciones más ricas e informativas, habilitando análisis de sentimiento y extracción de estrategias.

12. Conclusiones

Este trabajo ha presentado el diseño, implementación y evaluación experimental de un sistema completo de planificación de trayectorias profesionales con integración de LLM. Los resultados de 1.500 ejecuciones sobre 35 instancias permiten extraer conclusiones sólidas sobre el impacto de cada rol asignado al modelo de lenguaje.

1. Las variantes B y C no alteran la calidad de las trayectorias.

La interpretación LLM (B) y la evaluación post-hoc (C) producen trayectorias con coste idéntico a la línea base A en todos los tamaños (Wilcoxon $p = 1,0$, efecto = 0,0 créditos). El LLM en estos roles es neutro en cuanto a eficiencia.

2. La guía LLM paso a paso (D) incrementa el coste de forma significativa.

A vs D es significativo con FDR en los cuatro tamaños ($p < 3,3 \times 10^{-4}$). El efecto mediano va de +4,5 créditos en instancias small hasta +21 créditos en instancias large, producido por la tendencia del LLM a elegir por relevancia semántica en lugar de por eficiencia combinatoria.

3. La variante D es 200 veces más lenta en mediana.

La mediana de tiempo de D es 0,0756 s frente a 0,00038 s de A/B/C, con outliers de hasta 300 s en instancias grandes. El LLM como guía paso a paso no es viable en aplicaciones con requisitos de latencia estrictos.

4. La guía LLM mejora la tasa de éxito a costa del coste.

D alcanza una tasa de éxito del 92 % frente al 85,3 % de A/B/C, llegando al 100 % en instancias medium y large. Este trade-off puede ser aceptable cuando la cobertura del objetivo tiene mayor prioridad que la eficiencia de créditos.

5. El LLM pequeño no es un evaluador cuantitativo fiable.

La correlación entre la nota asignada por qwen2.5:3b y el coste real no es estadísticamente significativa (Pearson $r = -0,080$, $p = 0,161$). Las notas se concentran en 0–1 de una escala de 0–10, lo que indica que el modelo no comprende la tarea de evaluación de eficiencia.

6. La heurística A* ofrece el mejor equilibrio calidad-velocidad.

A* produce trayectorias con coste medio 19,19 créditos (vs 21,30 de greedy), una tasa de éxito del 93,3 % y un tiempo mediano de 4,3 ms. Es la opción recomendada para el planificador base.

ANEXO A — Parámetros completos del generador de instancias

El módulo `instance_generator.py` genera instancias sintéticas mediante el siguiente procedimiento: (1) crear un pool de habilidades de tamaño $\max(n_courses//5, 10)$, (2) asignar a cada curso entre 1 y 2 habilidades del pool garantizando que todas las habilidades sean concedidas por al menos un curso, (3) añadir prerequisites de curso eligiendo aleatoriamente entre los cursos ya creados (lo que garantiza la ausencia de ciclos en la estructura de dependencias), (4) seleccionar las habilidades objetivo como una muestra aleatoria del pool.

Parámetro	Small	Medium	Large	Manual
n_courses	10	30	100	5–8
max_credits	6	8	10	5
max_prereqs_c	2	2	3	1–2
target_skill_size	3	4	5	1–3
num_variants	10	10	10	5
seed_offset	0	100	200	N/A

Tabla A.1. Parámetros del generador de instancias por categoría.

ANEXO B — Prompts completos del LLM

B.1. Prompt de interpretación de objetivos (Variante B)

Intento 1:

Extrae las habilidades profesionales deseadas de la siguiente frase. Responde únicamente un JSON: {"habilidades": [...]} sin texto adicional. Frase: {texto}

Intento 2 (si intento 1 falla):

Extrae las habilidades profesionales deseadas de la siguiente frase. Responde únicamente un JSON válido con la clave "habilidades". No incluyas texto adicional. Frase: {texto}

B.2. Prompt de evaluación de trayectorias (Variante C)

Evalúa la siguiente secuencia de cursos para alcanzar el objetivo {objetivo}. Cursos: {lista_de_cursos_con_nombres}. Devuelve únicamente un JSON: {"nota": float, "justificacion": "..."}.

B.3. Prompt de sugerencia de siguiente curso (Variante D)

¿Qué curso recomendarías a continuación para alcanzar {objetivo}? Cursos disponibles: {lista_con_id_y_nombre}. Responde únicamente un JSON con {"course_id": "...", "justificacion": "..."}.

B.4. Configuración del LLM (llm_config.json)

```
{  
  "model": "qwen2.5:3b",  
  "endpoint_local": "http://localhost:11434",  
  "temperature": 0.1,  
  "max_tokens": 256,  
  "request_timeout": 8.0,  
  "max_retries": 2,  
  "backoff_base": 0.5  
}
```